Invoice Management Web App: Final Plan

1. What we're building

A web app where a **business logs in**, manages its **clients** and **products**, and creates **GST invoices** for each client using the business's own invoice layout. It replaces the manual Excel process. Target: a working demo in about 1.5 days.

2. Tech stack

- **Frontend:** HTML, CSS, JavaScript (`fetch()` to call the backend)
- **Backend:** Node.js + Express
- **Database:** MySQL (`mysql2` package)
- **Login:** `bcryptjs` (password hashing) + `express-session`
- **Tools:** VS Code, Postman, Git
- **To confirm:** Java was earlier said to be mandatory. Confirm that a Node.js backend is acceptable for the project.

3. Business rules

Structure

- One business has many clients. Each client has many invoices. Each invoice has one or more product lines.
- Each business has its own login (`login_id` + password). Passwords are stored only as hashes. Every query is filtered by the `business_id` stored in the session.

Products

- After logging in, the business adds its products once on a Products page: name, HSN code, unit, MRP, its own rate, GST%.
- MRP is the same for a product everywhere. The rate is set by each business.

Creating an invoice

1. Pick a client.
2. Add one or more products. For each, type only the quantity. Everything else is filled from the products table.
3. The app calculates and saves the invoice.

Calculations

- Amount = quantity x rate. GST is added on top of the amount.
- MRP is printed on the bill but not used in any calculation.
- Same state code for seller and buyer: CGST + SGST (half of the GST% each). Different states: IGST (full GST%).
- Round off to the nearest rupee, then total.
- Sample check: 120 x 62 = 7440; GST 5% = 372 (CGST 186 + SGST 186); total 7812.

Invoice number

- Format: `PREFIX/sequence/FY`, for example `SPI/24/2026-27`. No zero-padding.
- It works like a shared counter (like a Java `static` variable) and restarts at 1 each financial year (April to March).
- The counter lives in the database (`invoice_counters`), not in program memory. It is incremented inside a transaction so two invoices never get the same number.
- `UNIQUE (business_id, invoice_no)` blocks duplicates. GST rules allow at most 16 characters.

Per-client records

- All invoices of a client: `SELECT * FROM invoices WHERE client_id = ?`
- Invoice count per client: `SELECT client_id, COUNT(*) FROM invoices GROUP BY client_id`. This is calculated, never stored.

Layouts

- Each business has a saved layout (JSON) in `templates`: which header fields and item columns to show, in what order, with what labels.
- The layout controls only how the invoice looks. Calculations always use the fixed columns.
- Business-specific values (such as batch no. and packing) go in `extra` JSON columns.

4. Invoice contents (from the sample bill)

- **Seller:** name, address, DL no., GSTIN, state and state code, CIN, FSSAI, PAN
- **Buyer:** name, address, GSTIN, state and state code
- **Header fields:** invoice no., date, payment terms, delivery note, reference no., buyer's order no., dispatch doc no., dispatched through, destination, terms of delivery
- **Item columns:** description, HSN, packing, MRP, batch no., quantity, rate, per (unit), GST%, amount
- **HSN summary table:** lines grouped by HSN with taxable value, CGST and SGST
- **Footer:** round off, total, amount in words, declaration, signatory

5. Database tables

Table	Key columns
`businesses`	id, name, address, gstin, state, state_code, dl_no, cin, pan, fssai, invoice_prefix, login_id, password_hash
`clients`	id, business_id, name, address, gstin, state, state_code, phone
`products`	id, business_id, name, hsn_code, unit, mrp, rate, gst_rate, extra
`templates`	id, business_id, name, layout (JSON)
`invoice_counters`	business_id, fy, last_no (primary key on business_id + fy)
`invoices`	id, business_id, client_id, template_id, invoice_no, fy, invoice_date, due_date, status, subtotal, cgst, sgst, igst, round_off, total, extra
`invoice_items`	id, invoice_id, product_id, description, hsn_code, qty, unit, rate, mrp, gst_rate, amount, extra

`deals`	id, client_id, title, amount, status, deal_date (optional, build last)

Notes:

- Money columns use `DECIMAL(10,2)`, never `FLOAT`.
- `invoice_items` copies rate, MRP and GST% from the product so old bills never change when prices change.
- If you added `stock_qty` to `products`, drop it: `ALTER TABLE products DROP COLUMN stock_qty;`

6. Features

Must have: business login, products page, clients (add/view/edit), create invoice with several products, automatic numbering, CGST/SGST/IGST calculation, printable invoice (`window.print()`), pending/paid status, per-client invoice list and count, HSN summary.

Cut if time runs short: dashboard, deals, Excel upload for layouts, PDF libraries.

7. Remaining build plan

1. Set up the Node project, connect to MySQL, test with a simple route.
2. Business signup/login with bcrypt and sessions, plus a middleware that protects routes.
3. Clients routes and page.
4. Products routes and page.
5. Layout selection and the invoice creation screen (add several product rows).
6. Invoice saving: counter in a transaction, calculations, items.
7. Printable invoice view, status update, per-client list and count.
8. Testing and demo data.

8. Good practices

- Serve the HTML pages from Express (`express.static('public')`) to avoid CORS and cookie problems.
- Use parameterized queries (`WHERE id = ?`), never build SQL by joining text.
- Keep the database password in a `.env` file and never upload it to GitHub.
- Write the core code yourself and use AI as a tutor for explaining errors and reviewing your code.